

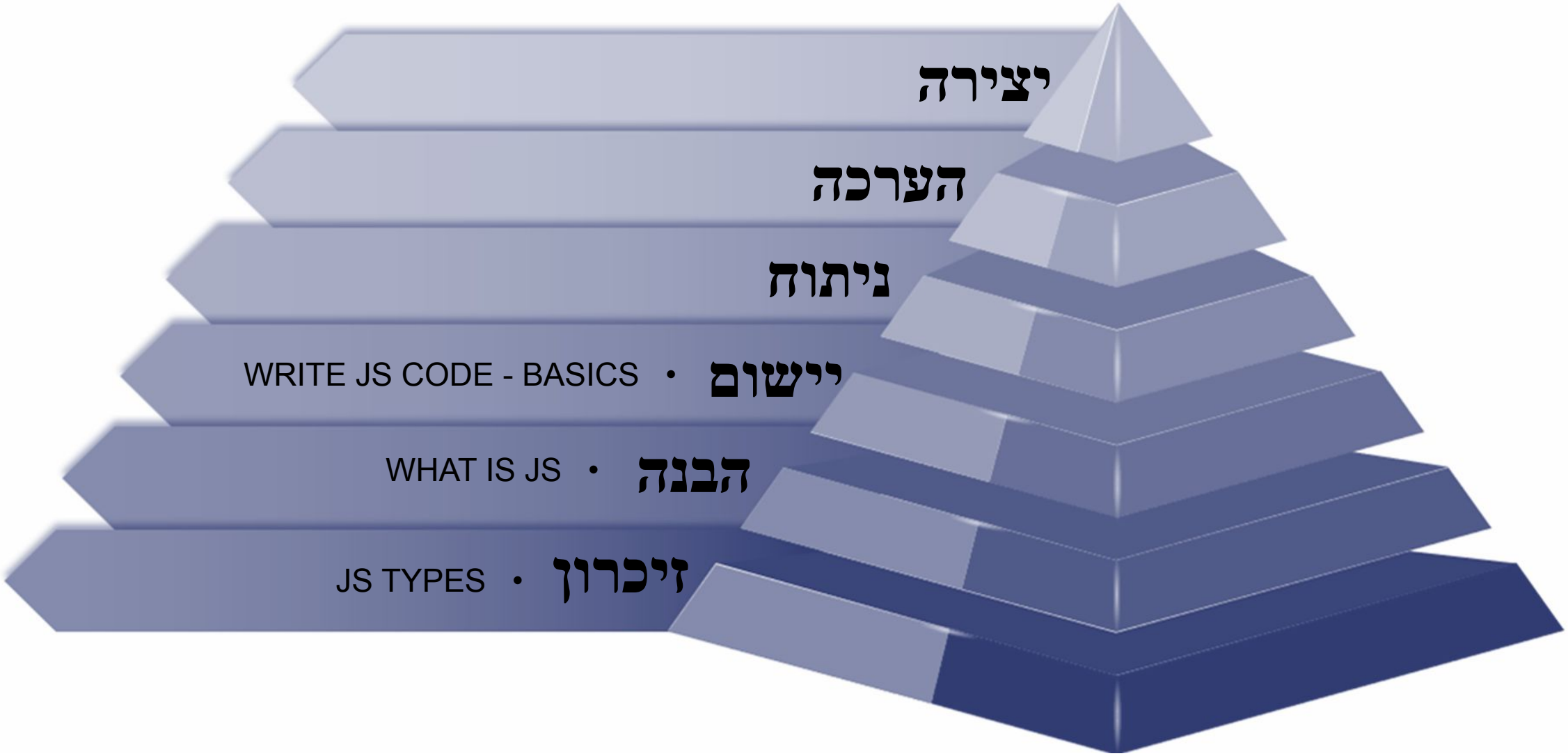
KODCODE

JS BASICS

תוכן העניינים

- Goal: understand js basics
- vars and types
- js operations between types

מטרות השיעור





KODCODE

js intro

js intro

- Welcome to web development!
- You start a new Journey!

js intro



js intro

- but not really…



js intro

- JS is just another programming language…



js intro

- Most of what you know is pretty much the same
- other than syntax differences
- so let’s learn a bit about js

js intro

- Js was created in 1995 by Brendan Eich.
- The original purpose was to add reactivity to the web.
- Today, it’s used for everything.

js intro



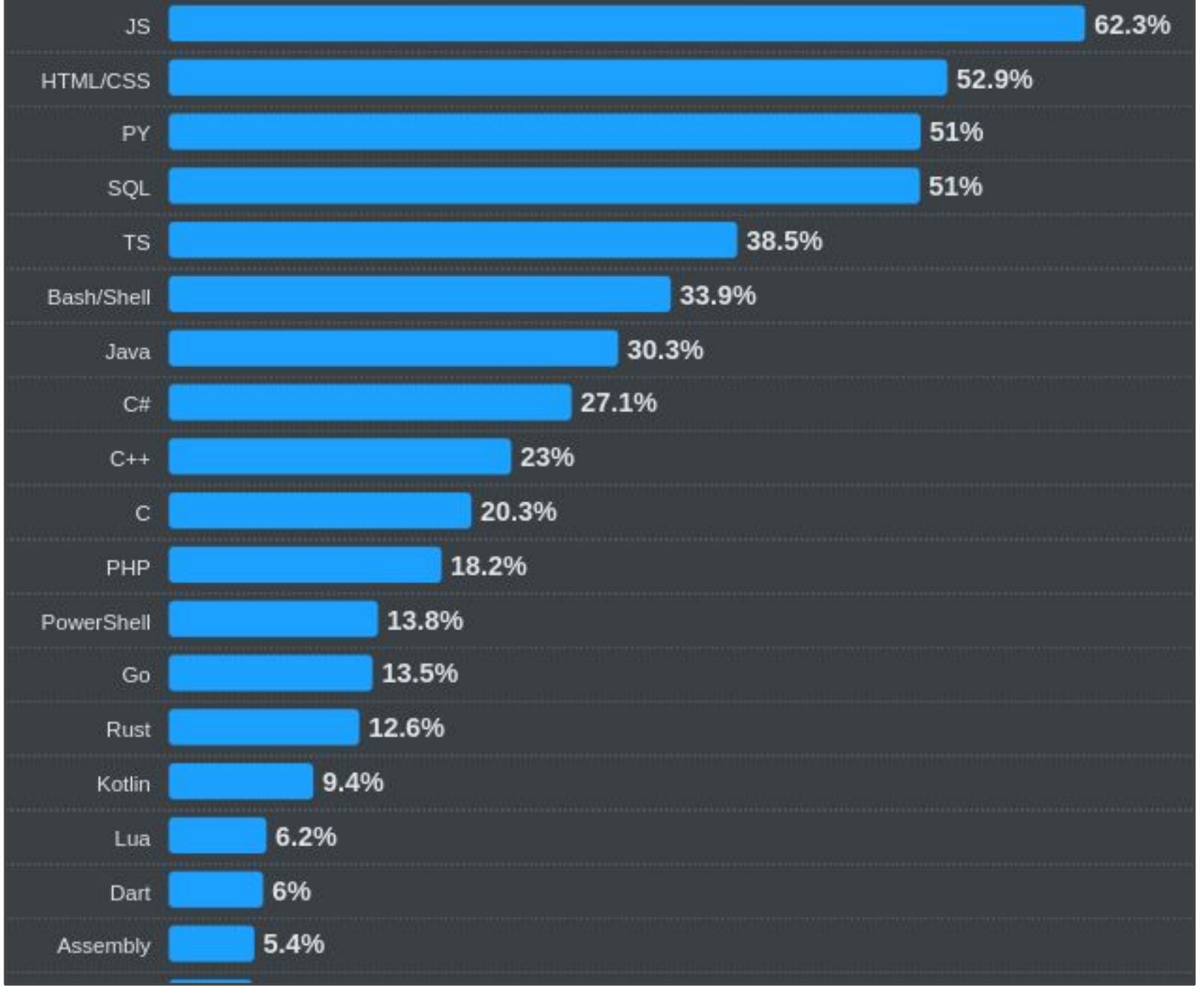
“Every application that can be written with JS, will eventually be built with JS”

—Jeff Atwood, creator of stackoverflow

js intro

- BY FAR the most popular programming language today.

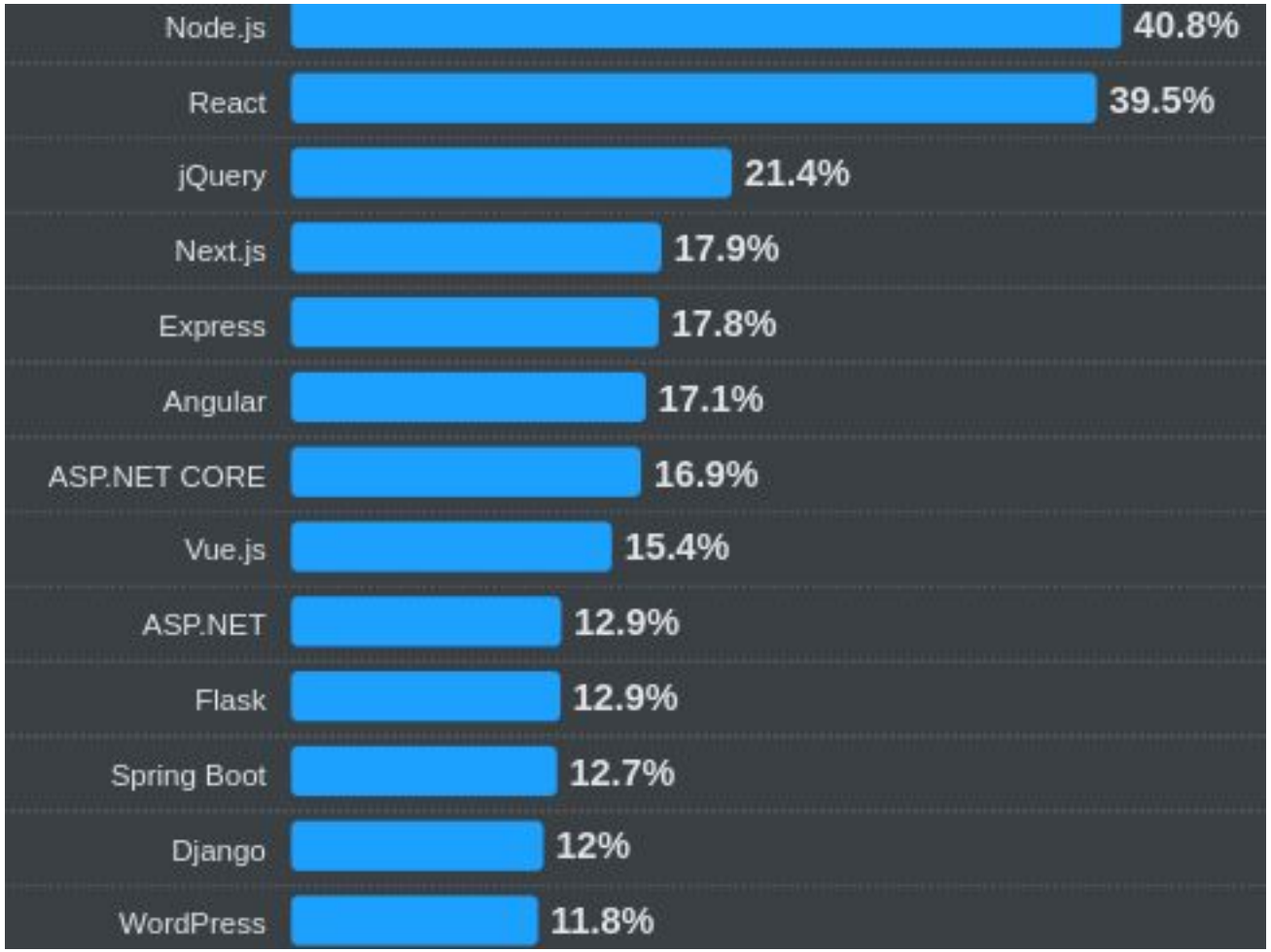
Source



js intro

- In backend too!

Source





KODCODE

js vars and types

js vars and types

- There are 2 main ways to run js:
  - in the browser
  - in the terminal

- we will focus on the second part more
- for that - install nodejs

js vars and types

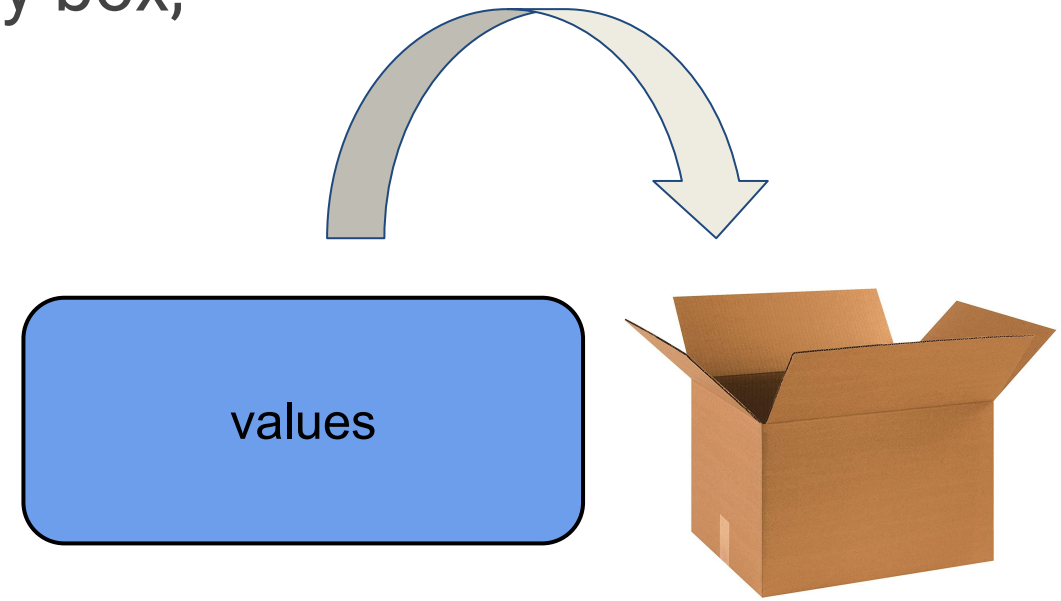
- we run js code with node in the terminal by:
  - create .js file
  - running: node /path/to/file/<file name>.js

js vars and types

- task:
  - create some file with .js extension
  - inside, write: console.log(“hello world”);
  - run it with nodejs - see the output.

js vars and types

- The core tool we use in every programming language is a **variable**. What is it?
- You can think of it as an empty box, That you can put values inside.



js vars and types

- Creating a variable is called: **declaration**.
- Giving a value to a variable is called: **assigning**.
- How do we declare a variable and assign a value in JS?
- The basic structure is:

  Keyword varName = value;

- Keyword + varName = **declaration**.
  = + value = **assigning**.

js vars and types

- So to summarize:



js vars and types

- Keyword varName = value;
  a. Keyword: a reserved word the the js engine understands - to tell that we are creating a variable.
  b. varName: any name you like, cannot be a number.
  c. Value: any value that js knows. We will talk about what value type are there.

js vars and types

- So if we want to declare a variable, we have 3 possible keywords: **var**, **let** and **const**.
- For example:
  a. var myName = ‘yishai’
  b. let myName = ‘yishai’
  c. const myName = ‘yishi’
- What is the difference?

js vars and types

- **Var** - this is the old way to declare variables. Never use it yourself.
- **Let** - signals that this variable can change its value.
- **Const** - signals that this variable cannot change its value.

js vars and types

- When we say “change its value”, what do we mean?
- Remember that a variable is like a box. Even if you put something inside, sometimes we can open in again, take everything out and put inside something new.
- How do we do this in js?

js vars and types

- If we declared a variable like so:
  var name = ‘yishai’

  We can **reassign** it with:
  name = ‘bob’

  We don’t need to **redeclare**! (i.e. use the keyword again)
  This will results in an error:

  var name = ‘alice”;

js vars and types

- Reassigning only works if we declared a variable with “var” or “let”.
- If we use “const” to declare a variable - the variable is constant, and it cannot be reassigned.
- This will be an error:
  const name = ‘yishai’
  name = ‘bob’

js vars and types

- Which declaration keyword would we prefer?
- why?

js vars and types

- For me (and thus - for you):
- CONST IS DEFAULT
- Use let only if you are SURE that the value is going to change.
- NEVER use var.

js vars and types

- How can we see our variables?
- Lets learn our first js function (more on functions later): console.log(value).

  Running this command will print what we put inside the () to the console.
- For example: console.log(‘yishai’) // prints yishai

js vars and types

- We saw declarations and assignments, now let us talk about the values.
- What values can we put inside a variables in JS?
- There are many types, we will start with the **primitive** types.

js vars and types

- The important primitive types are:
  a. string (text)
  b. number (int and float)
  c. boolean (true/false)
  d. undefined
  e. null
- we can see what type we are using by using **typeof** operator.
  console.log(typeof <var name>);

js vars and types

- Strings:
  a. If we want a variable to have a value of string, we declare it in 3 ways:
     i. Single quotes: ‘yishai’
     ii. Double quotes: “yishai”
     iii. Backticks: `yishai`
  b. They are all the same in JS. backticks have more properties.

js vars and types

- Strings:
  a. We can combine other variables inside a string using backticks.
  b. This is called “string interpolation”.
     const age = 35
     console.log(`my age is ${age}`);
     Inside of:
     console.log(“my age is”,age);

js vars and types

- Numbers:
  a. in JS it doesn’t matter if its a natural number or a decimal number, all the numbers are the same.
  b. 4 and 4.0 are the same, both are considered numbers.

js vars and types

- Boolean:
  a. Named after the mathematician George Boole.
  b. Can have one of two values: true or false:
  c. Important: true != ‘true’. Boolean is not a string!

js vars and types

- Null/undefined:
  a. Used to signal that there is no value.
  b. Undefined is used to signal that the variable exists, but it’s not assigned a value.
  c. Null is used to signal that there is no variable at all.
  d. Both can be used, pretty much interchangeably.

js vars and types

- So we declared a variable and assigned a value.
- What can we do with it now?

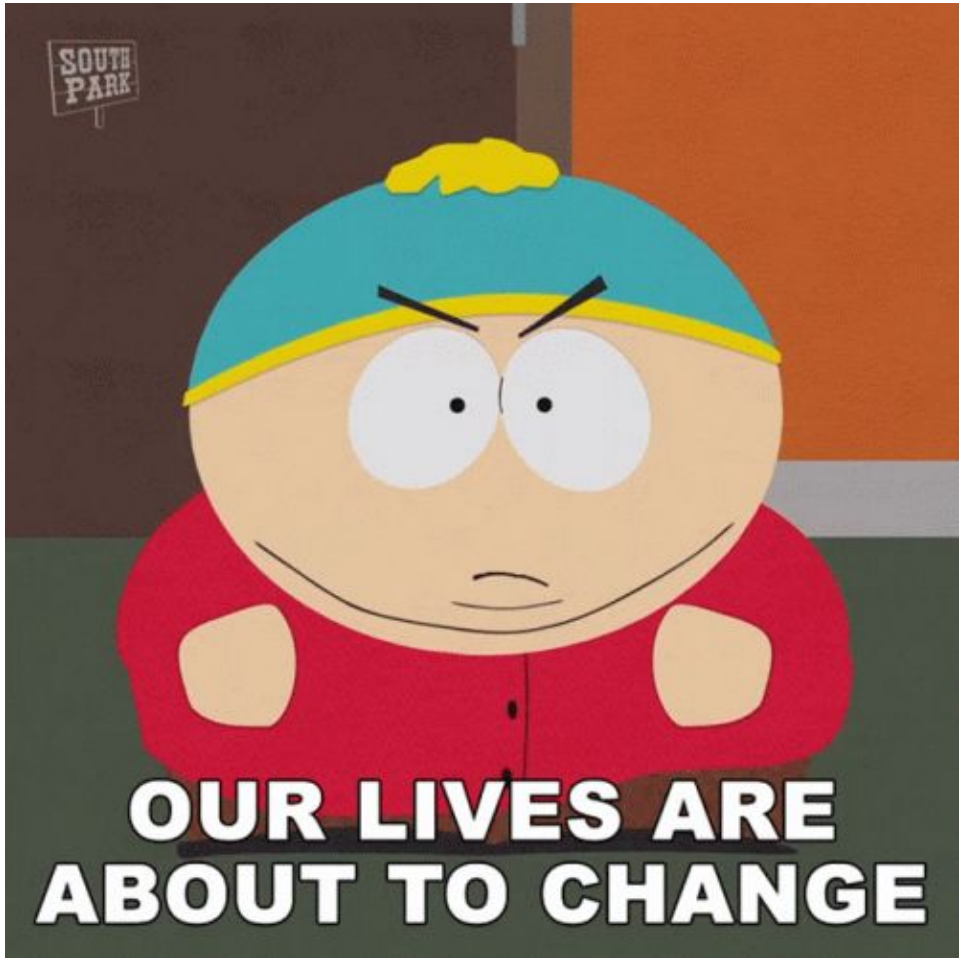
js vars and types

- Move it around!



js vars and types

- Change it!



js vars and types

- Short exe:
  a. Create a js program to show your basic data.
  b. Create variables for:
     i. Age
     ii. Name
     iii. Address
     iv. Are you vegan
     v. Number of kids
  c. Print them all together with string interpolation.
  d. Print all of the types of your variables



KODCODE

js operations

js vars and types

- Operations between types:
  a. What happens in JS if you try to do:
     34 + “5”. What will be the result?
  b. Try it with diff primitive types and with diff mathematical operations.

js vars and types

- In general:
  + with string - will change everything into string.
- If there is a boolean - it will turn the boolean to 1 or 0 for true or false, respectively.
- Math operations that are not + between strings and others - will result in NAN - not a number.